

FINAL C++ NOTES

C++ Basics → Programming Fundamentals → DSA Preparation

Simple, practical notes for learning C++ from scratch.

These notes focus on understanding the syntax, remembering the important ideas, and writing practice programs without unnecessary complexity.

Keep these notes together with your practice programs. The goal is not to memorize every line of code — understand the idea, then practice writing it yourself.

1. What is C++?

C++ is a general-purpose programming language used for software development, games, competitive programming, systems programming, and Data Structures & Algorithms.

First C++ Program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World";
    return 0;
}
```

Explanation:

- `#include <iostream>` → provides input/output features such as `cin` and `cout`.
- `using namespace std;` → lets us use standard library names without repeatedly writing `std::`.
- `main()` → program execution starts here.
- `cout` → displays output.
- `return 0` → indicates successful program completion.

2. Comments

Comments are ignored by the compiler and are used to explain code.

```
// Single-line comment

/*
    Multi-line
    comment
*/
```

3. Variables

A variable is a named memory location used to store a value.

```
int age = 18;
```

Declaration:

```
int age;
```

Initialization:

```
int age = 18;
```

Changing a value:

```
age = 20;
```

4. Data Types

Data Type	Meaning	Example
int	Integer	10
float	Decimal	10.5
double	More precise decimal	10.5678
char	Single character	'A'
bool	True / False	true

string	Text	"Hello"
--------	------	---------

```
int age = 18;
float marks = 89.5;
double pi = 3.14159;
char grade = 'A';
bool pass = true;
string name = "Sujith";
```

5. Input and Output

cout — Output

```
cout << "Hello";
cout << "Age = " << age;
cout << "Hello\n";
```

cin — Input

```
int age;
cin >> age;

int a, b;
cin >> a >> b;
```

Example

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << a + b;
    return 0;
}
```

6. Operators

Arithmetic Operators

+ Addition - Subtraction * Multiplication / Division % Remainder

```
int a = 10;
int b = 3;

cout << a + b; // 13
cout << a - b; // 7
cout << a * b; // 30
cout << a / b; // 3
cout << a % b; // 1
```

Integer Division

```
cout << 5 / 2;   // 2
cout << 5.0 / 2; // 2.5
```

Increment and Decrement

```
x++; // increase by 1
x--; // decrease by 1
++x; // prefix increment
x++; // postfix increment
```

Prefix changes the value before it is used. Postfix uses the old value first, then changes it.

Relational Operators

> greater than < less than >= greater/equal <= less/equal == equal != not equal

Logical Operators

&& AND → both conditions must be true.

|| OR → at least one condition must be true.

! NOT → reverses the result.

Assignment Operators

```
x = 10;
x += 5; // x = x + 5
x -= 5; // x = x - 5
x *= 5; // x = x * 5
x /= 5; // x = x / 5
x %= 5; // x = x % 5
```

7. Conditional Statements

if

```
if (condition) {
    // code
}
```

if-else

```
int n;
cin >> n;

if (n % 2 == 0) {
    cout << "Even";
}
else {
    cout << "Odd";
}
```

if-else-if

```
if (marks >= 90) {
    cout << "A";
}
else if (marks >= 80) {
    cout << "B";
}
else if (marks >= 70) {
    cout << "C";
}
else {
    cout << "D";
}
```

Conditions are checked from top to bottom.

Nested if

```
if (age >= 18) {
    if (hasID) {
        cout << "Allowed";
    }
}
```

switch

```
switch(day) {
    case 1:
        cout << "Monday";
        break;

    case 2:
        cout << "Tuesday";
}
```

```

        break;

    default:
        cout << "Invalid";
}

```

break stops the switch. default runs when no case matches.

8. Loops

Loops are used to repeat a block of code. The main loops are for, while, and do-while.

for Loop

```

for (initialization; condition; update) {
    // code
}

for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}

```

Easy way to remember: initialize → check → execute → update → repeat.

while Loop

```

int i = 1;

while (i <= 5) {
    cout << i << " ";
    i++;
}

```

Remember to update the loop variable when necessary, otherwise an infinite loop can occur.

do-while Loop

```

int i = 1;

do {
    cout << i << " ";
    i++;
} while (i <= 5);

```

The do-while loop executes its body at least once because the condition is checked at the end.

break

```

for (int i = 1; i <= 10; i++) {
    if (i == 5)
        break;
    cout << i << " ";
}

```

Immediately terminates the loop.

continue

```

for (int i = 1; i <= 5; i++) {
    if (i == 3)
        continue;
    cout << i << " ";
}

```

Skips the current iteration.

9. Pattern Printing

Pattern questions mainly use nested loops.

Rule: Outer loop → rows. Inner loop → columns/items.

Increasing Star Pattern

```
*
**
***
****

for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Decreasing Star Pattern

```
for (int i = 4; i >= 1; i--) {
    for (int j = 1; j <= i; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Number Pattern

```
1
12
123
1234

for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        cout << j;
    }
    cout << endl;
}
```

10. Functions

A function is a reusable block of code that performs a specific task.

```
returnType functionName(parameters) {
    // code
}
```

Example

```
int add(int a, int b) {
    return a + b;
}
```

```
cout << add(5, 3);
```

int → return type add → function name a, b → parameters

void Function

```
void greet() {
    cout << "Hello";
}
```

```
greet();
```

A void function does not return a value.

Pass by Value

```
void change(int x) {
    x = 100;
```

```

}

int a = 10;
change(a);
cout << a; // 10

```

A copy of the value is passed, so the original variable is not changed.

Pass by Reference

```

void change(int &x) {
    x = 100;
}

int a = 10;
change(a);
cout << a; // 100

```

Using & here lets the function work with the original variable.

11. Pointers

A pointer is a variable that stores the address of another variable.

```

int x = 10;
int *p = &x;

```

x → stores 10

&x; → address of x

p → stores the address of x

*p → value at that address

Address Operator

```

int x = 10;
cout << &x;

```

Dereference Operator

```

int x = 10;
int *p = &x;

cout << *p; // 10

```

Memory trick: & → address, * → value at address.

12. Recursion

Recursion means a function calls itself.

```

void print(int n) {
    if (n == 0)
        return;

    cout << n << " ";
    print(n - 1);
}

```

Calling print(5) gives: 5 4 3 2 1.

Base Case

The condition that stops recursion is called the base case. Every recursive function needs a proper stopping condition.

Factorial Using Recursion

```

int factorial(int n) {
    if (n == 0 || n == 1)

```

```

        return 1;

    return n * factorial(n - 1);
}

```

For example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

13. Arrays

An array stores multiple values of the same type.

```
int arr[5] = {10, 20, 30, 40, 50};
```

Array Indexing

C++ uses zero-based indexing.

```

Index:  0  1  2  3  4
Value: 10 20 30 40 50

```

For an array of size n , valid indexes are 0 to $n-1$.

Traversing an Array

```

int arr[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; i++) {
    cout << arr[i] << " ";
}

```

Maximum Element

```

int arr[5] = {10, 40, 20, 90, 30};

int maximum = arr[0];

for (int i = 1; i < 5; i++) {
    if (arr[i] > maximum) {
        maximum = arr[i];
    }
}

cout << maximum;

```

Linear Search

```

int arr[5] = {10, 20, 30, 40, 50};
int target = 30;

bool found = false;

for (int i = 0; i < 5; i++) {
    if (arr[i] == target) {
        found = true;
        break;
    }
}

if (found)
    cout << "Found";
else
    cout << "Not Found";

```

Sum of Array

```

int arr[5] = {10, 20, 30, 40, 50};
int sum = 0;

for (int i = 0; i < 5; i++) {
    sum += arr[i];
}

```

```

}

cout << sum;

```

Reverse an Array

```

int arr[5] = {1, 2, 3, 4, 5};

int i = 0;
int j = 4;

while (i < j) {
    swap(arr[i], arr[j]);
    i++;
    j--;
}

```

14. 2D Arrays

A 2D array is basically a table or matrix.

```

int arr[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

```

Think of it as rows × columns.

Traversing a 2D Array

```

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}

```

Outer loop → rows

Inner loop → columns

15. Strings

A string is a sequence of characters.

```

string name = "Sujith";

```

String Input

```

string name;
cin >> name;    // one word

getline(cin, name); // full line / sentence

```

getline() can read spaces.

String Indexing

```

string s = "Hello";

```

```

H e l l o
0 1 2 3 4

```

So s[0] is H and s[4] is o.

String Length

```

string s = "Hello";

cout << s.length();
cout << s.size();

```

Both give 5 for "Hello".

Traverse a String

```
string s = "Hello";

for (int i = 0; i < s.length(); i++) {
    cout << s[i] << " ";
}
```

Palindrome

A palindrome reads the same forward and backward, e.g. madam, level, racecar.

```
bool isPalindrome(string s) {
    int i = 0;
    int j = s.length() - 1;

    while (i < j) {
        if (s[i] != s[j]) {
            return false;
        }
        i++;
        j--;
    }

    return true;
}
```

Useful Character Functions

```
#include <cctype>

isdigit(c)
isalpha(c)
islower(c)
isupper(c)
tolower(c)
toupper(c)
```

16. Important Things to Remember

= vs ==

```
x = 5; // assignment
x == 5; // comparison
```

Array indexing: for size $n \rightarrow 0$ to $n-1$.

Integer division: $5 / 2$ gives 2 when both operands are integers.

break: exits the loop or switch.

continue: skips the current iteration.

&: address operator / reference symbol depending on context.

*: dereference operator when used with a pointer.

Recursion: function calling itself; always identify the base case.

17. Practice Programs

Basic

- ☐ Sum of two numbers
- ☐ Even or odd
- ☐ Positive, negative or zero

- ☐ Maximum of two numbers
- ☐ Maximum of three numbers
- ☐ Swap two numbers
- ☐ Simple calculator
- ☐ Area of circle
- ☐ Area of rectangle
- ☐ Check leap year

Loops

- ☐ Print 1 to N
- ☐ Print N to 1
- ☐ Sum 1 to N
- ☐ Sum of even numbers
- ☐ Multiplication table
- ☐ Count digits
- ☐ Sum of digits
- ☐ Reverse a number
- ☐ Palindrome number
- ☐ Factorial
- ☐ Prime number
- ☐ Armstrong number

Patterns

- ☐ Square
- ☐ Rectangle
- ☐ Increasing triangle
- ☐ Decreasing triangle
- ☐ Number triangle
- ☐ Reverse number triangle
- ☐ Pyramid
- ☐ Inverted pyramid

Functions

- ☐ Addition function
- ☐ Maximum function
- ☐ Factorial function
- ☐ Prime function
- ☐ Palindrome function
- ☐ Swap using reference
- ☐ Swap using pointer

Recursion

- ☐ Print 1 to N
- ☐ Print N to 1

- ☐ Factorial
- ☐ Sum of N numbers
- ☐ Fibonacci
- ☐ Power
- ☐ Reverse string
- ☐ Palindrome

Arrays

- ☐ Input/output
- ☐ Sum
- ☐ Maximum
- ☐ Minimum
- ☐ Linear search
- ☐ Reverse
- ☐ Count even/odd
- ☐ Second largest
- ☐ Check sorted array
- ☐ Frequency

2D Arrays

- ☐ Matrix input/output
- ☐ Matrix addition
- ☐ Matrix subtraction
- ☐ Transpose
- ☐ Row sum
- ☐ Column sum
- ☐ Diagonal sum
- ☐ Matrix search

Strings

- ☐ Length
- ☐ Reverse
- ☐ Palindrome
- ☐ Count vowels
- ☐ Count consonants
- ☐ Count digits
- ☐ Character frequency
- ☐ Uppercase/lowercase
- ☐ Remove spaces

18. One-Page Revision

cout → output
cin → input

if → decision

else → alternative

for → repetition

while → repetition

do-while → executes at least once

break → exit

continue → skip current iteration

function → reusable block of code

&x → address of x

*p → value stored at address p

array → collection of same type

string → sequence of characters

recursion → function calling itself

Array of size n:

valid indexes = 0 to n-1

Best way to use these notes: Read the concept → close the notes → write the program yourself → test it → change the input and test again. Understanding + practice beats memorizing code.